

23070521054- Gauri Gulhane

```

# =====
# PRACTICAL 4
# Regularization Techniques for Deep Learning
# L1, L2 Regularization and Dropout
# =====

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.regularizers import l1, l2

# -----
# 1. Load Housing Dataset
# -----

data = fetch_california_housing()

X = data.data
y = data.target

print("Dataset Shape:", X.shape)

# -----
# 2. Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# -----
# 3. Feature Scaling
# -----

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# 4. Function to create model
# -----

def create_model(model_type):

    if model_type == "Baseline":

        model = Sequential([
            Input(shape=(X_train.shape[1],)),
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(32, activation='relu'),
            Dense(1)
        ])

    elif model_type == "L1":

        model = Sequential([
            Input(shape=(X_train.shape[1],)),
            Dense(
                128,
                activation='relu'
            )
        ])

```

```

        activation='relu',
        kernel_regularizer=l1(0.001)
    ),
    Dense(
        64,
        activation='relu',
        kernel_regularizer=l1(0.001)
    ),
    Dense(32, activation='relu'),
    Dense(1)
])

elif model_type == "L2":

    model = Sequential([
        Input(shape=(X_train.shape[1],)),
        Dense(
            128,
            activation='relu',
            kernel_regularizer=l2(0.001)
        ),
        Dense(
            64,
            activation='relu',
            kernel_regularizer=l2(0.001)
        ),
        Dense(32, activation='relu'),
        Dense(1)
    ])

elif model_type == "Dropout":

    model = Sequential([
        Input(shape=(X_train.shape[1],)),

        Dense(128, activation='relu'),
        Dropout(0.30),

        Dense(64, activation='relu'),
        Dropout(0.30),

        Dense(32, activation='relu'),

        Dense(1)
    ])

    model.compile(
        optimizer='adam',
        loss='mse',
        metrics=['mae']
    )

    return model

# -----
# 5. Train Different Models
# -----

model_types = [
    "Baseline",
    "L1",
    "L2",
    "Dropout"
]

results = {}
histories = {}

for model_type in model_types:

    print("\n=====")
    print("Training:", model_type)
    print("=====")

    model = create_model(model_type)

    history = model.fit(

```

```

        X_train,
        y_train,
        validation_split=0.20,
        epochs=20,
        batch_size=64,
        verbose=0
    )

    prediction = model.predict(
        X_test,
        verbose=0
    ).flatten()

    rmse = np.sqrt(
        mean_squared_error(
            y_test,
            prediction
        )
    )

    results[model_type] = rmse
    histories[model_type] = history

    print(
        model_type,
        "RMSE:",
        round(rmse, 4)
    )

# -----
# 6. Compare Results
# -----

print("\nFINAL COMPARISON")
print("-----")

for model_type, rmse in results.items():

    print(
        model_type,
        "RMSE =",
        round(rmse, 4)
    )

# -----
# 7. Plot Validation Loss
# -----

plt.figure(figsize=(10, 6))

for model_type in model_types:

    plt.plot(
        histories[model_type].history['val_loss'],
        label=model_type
    )

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")

plt.title(
    "Comparison of Regularization Techniques"
)

plt.legend()
plt.show()

```

```
Dataset Shape: (20640, 8)

=====
Training: Baseline
=====
Baseline RMSE: 0.539

=====
Training: L1
=====
L1 RMSE: 0.5487

=====
Training: L2
=====
L2 RMSE: 0.5446

=====
Training: Dropout
=====
Dropout RMSE: 0.5526

FINAL COMPARISON
-----
Baseline RMSE = 0.539
L1 RMSE = 0.5487
L2 RMSE = 0.5446
Dropout RMSE = 0.5526
```

